

DESENVOLVIMENTO MOBILE

AULA 4 – PRESSABLE, FLATLIST E SECTIONLIST

CESAR School · 2026.2



NA AULA 3 EU TE DEI UM **REMENDO**.

O **Button** NÃO ACEITA **style**, ENTÃO USAMOS **<Text onPress>**. HOJE DEVOLVO O COMPONENTE DE VERDADE.



AGENDA – 3 HORAS

Bloco	Tempo
Abertura + o problema do dia	10 min
1. Pressable : o toque que você desenha	35 min
☕ Intervalo	10 min
2. Checkpoint – toque	8 min
3. FlatList : só renderiza o que se vê	42 min
4. SectionList : quando o dado vem agrupado	20 min
5. Checkpoint – listas	7 min
6. Hands-on guiado	38 min
Atividade aplicada + fechamento	10 min



AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

- 1 **Construir** um controle tocável com aparência própria e feedback visual
- 2 **Descrever** o ciclo de vida do toque e escolher o callback certo
- 3 **Ajustar** área e tempo do toque com `hitSlop` e companhia
- 4 **Explicar** por que uma lista longa não pode ser um `.map()` em `ScrollView`
- 5 **Renderizar** uma `FlatList` completa — chave, molduras, estado vazio
- 6 **Diagnosticar** os três defeitos clássicos de lista
- 7 **Escolher** entre `FlatList` e `SectionList` pelo formato do dado
- 8 **Prever** a diferença de sticky header entre iOS e Android

PARTE 1

PRESSABLE – O TOQUE QUE VOCÊ DESENHA



O QUE VOCÊ TINHA, E O LIMITE DE CADA UM

O que você tinha	O limite
Button	aparência do sistema; não aceita style
<Text onPress>	funciona, mas o alvo é só o texto e não há feedback
Switch	é um liga/desliga, não uma ação
Pressable	envolve qualquer coisa e reporta o toque

A FRASE-CHAVE: **Pressable NÃO DESENHA NADA.**





O TAPETE SENSOR

O TAPETE NÃO TEM APARÊNCIA PRÓPRIA E NÃO ABRE PORTA NENHUMA. ELE SÓ AVISA: "ALGUÉM PISOU AQUI".

O que acontece depois é decisão sua.

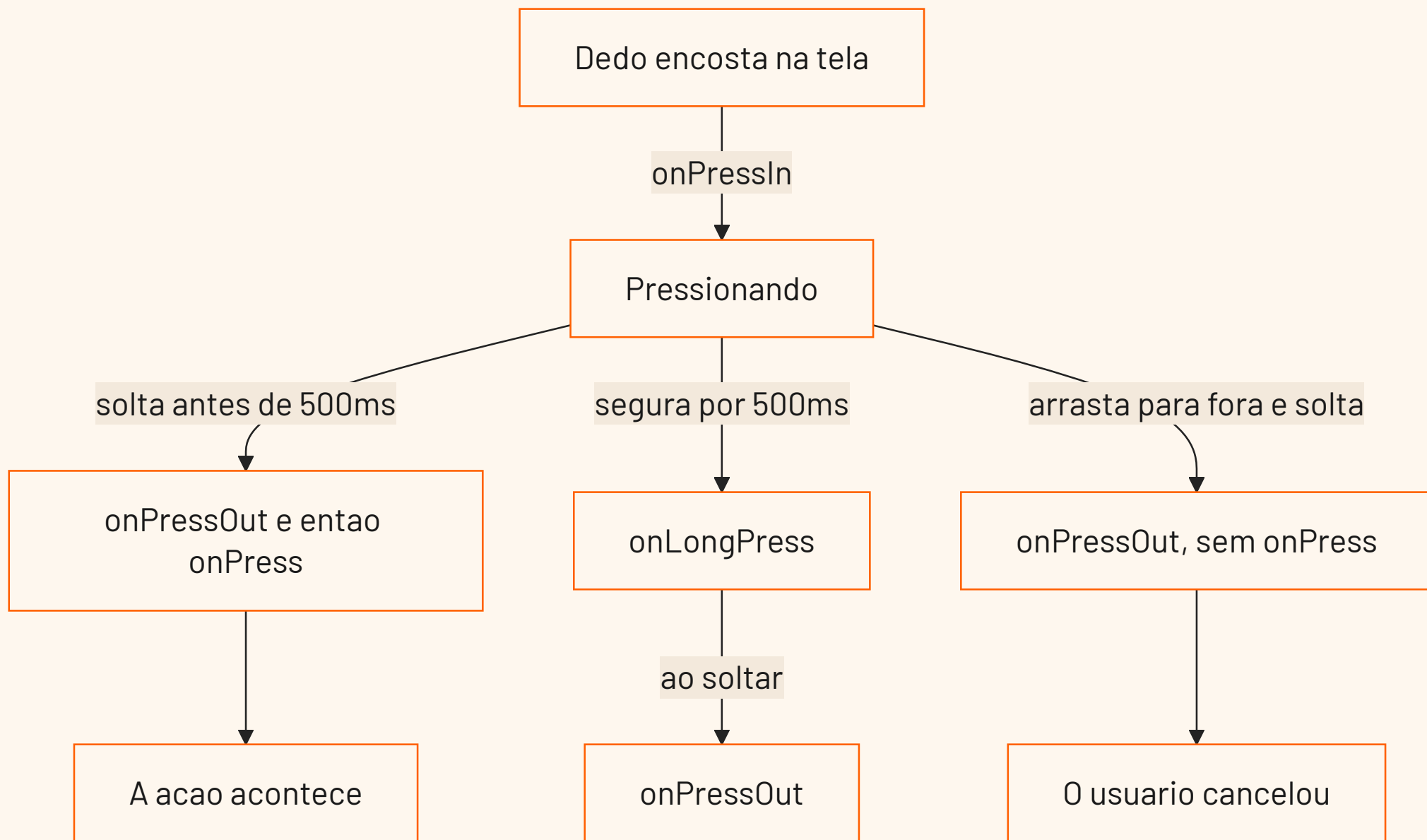
O MÍNIMO FUNCIONAL

```
<Pressable onPress={onSalvar} style={styles.botao}>
  <Text style={styles.rotulo}>Salvar hábito</Text>
</Pressable>

const styles = StyleSheet.create({
  botao: {
    paddingVertical: 14, paddingHorizontal: 20,
    borderRadius: 12, backgroundColor: '#FF6002', alignItems: 'center',
  },
  rotulo: { color: '#FFFFFF', fontSize: 16, fontWeight: '600' },
});
```

NADA AÍ É NOVO ALÉM DO **Pressable** .

O CICLO DE VIDA DO TOQUE





onPress DISPARA QUANDO O DEDO **SOLTA**.

AÇÃO VAI EM **onPress**. FEEDBACK VAI NO **pressed**.

QUAL CALLBACK USAR

Callback	Quando usar
onPress	a ação. Salvar, marcar, abrir, alternar
onPressIn	efeito instantâneo e reversível (som, vibração leve)
onPressOut	desfazer algo que começou no onPressIn
onLongPress	ação secundária: menu, excluir, seleção múltipla
onPressMove	arrastar; quase nunca na disciplina

A ÚNICA SINTAXE NOVA: **style** COMO FUNÇÃO

```
// Aula 3 – array com condicional vinda do SEU estado
<View style={[styles.card, concluido && styles.cardConcluido]} />

// Aula 4 – o MESMO array; quem decide agora é o framework
<Pressable style={({ pressed }) => [styles.card, pressed && styles.cardPressionado]}>
  <Text>Beber água</Text>
</Pressable>
```

MESMA PRECEDÊNCIA DA AULA 3: O ÚLTIMO VENCE, **false** É DESCARTADO.

O QUE ISSO SUBSTITUI

NA WEB

```
.card:active {  
  background: #F3E9DC;  
}
```

Pseudo-classe. O navegador aplica.

NÃO EXISTE `:active`. E NÃO EXISTE `:hover` – NÃO EXISTE MOUSE.

NO REACT NATIVE

```
style={({ pressed }) => [  
  styles.card,  
  pressed && styles.cardPressionado,  
]}
```

Parâmetro. Você aplica.

children **TAMBÉM ACEITA FUNÇÃO**

```
<Pressable onPress={marcarConcluido}>
  ({ pressed }) => <Text>{pressed ? 'Soltando...' : 'Marcar concluído'}</Text>
</Pressable>
```

Use quando o **conteúdo** muda, não só o estilo.

NA MAIORIA DOS CASOS, MUDAR O ESTILO BASTA.

hitSlop – O MAIS ÚTIL E O MENOS USADO

```
<Pressable onPress={remover} hitSlop={12}>  
  <Text style={styles.iconeRemover}>x</Text>  
</Pressable>
```

Aumenta a área **sensível** – **sem** mudar nada do layout.

- O erro correspondente: aumentar o `padding` "para o toque pegar".

ÁREA E TEMPO: O CATÁLOGO

Sintoma	Prop	Default
Alvo pequeno, o usuário erra	hitSlop	nenhum
O toque cancela se o dedo escorrega	pressRetentionOffset	{t:20, l:20, r:20, b:30}
O item pisca enquanto eu rolo	unstable_pressDelay	nenhum
500 ms de long press é muito/pouco	delayLongPress	500

unstable_pressDelay É A PROP QUE VOCÊ VAI QUERER NA PARTE 3.



ANDROID: `android_ripple`

```
<Pressable
  onPress={marcar}
  android_ripple={{ color: '#FFB132', borderless: false }}
  style={({ pressed }) => [styles.card, pressed && styles.cardPressionado]}
>
```

- No **iOS não faz nada** — e isso é o esperado, não um bug
- Aceita `PlatformColor` : a ondinha acompanha claro/escuro do sistema
- `android_disableSound` desliga o som do toque

PROP COM PREFIXO DE PLATAFORMA É UM CONTRATO HONESTO.

ACESSIBILIDADE EM DUAS PROPS

```
<Pressable
  onPress={marcarConcluido}
  accessibilityRole="button"
  accessibilityLabel="Marcar hábito Beber água como concluído"
  disabled={carregando}
>
```

- `accessibilityRole="button"` – sem isso, o leitor anuncia um agrupamento sem propósito
- `accessibilityLabel` – escreva a **ação**, não o desenho
- `disabled` – desliga o toque **e** informa o estado



O QUE VOCÊ VAI ENCONTRAR POR AÍ

Praticamente todo tutorial monta botões com **TouchableOpacity**.

A documentação oficial **do próprio TouchableOpacity** avisa, no topo da página:

"If you're looking for a more extensive and future-proof way to handle touch-based input, check out the **Pressable** API."

QUANDO ENCONTRAR UM EXEMPLO COM ELE: TROQUE PELO PRESSABLE.



CHECKPOINT 1

60 SEGUNDOS EM DUPLAS, E A GENTE RESPONDE EM VOZ ALTA.



CHECKPOINT 1 – AS PERGUNTAS

- 1 `onPressIn` × `onPress` : onde vai a ação de salvar, e por quê?
- 2 Ícone de 16×16 e o usuário erra o toque: `padding` ou outra coisa?
- 3 Escreva a assinatura do `style` que escurece o card enquanto pressionado.
- 4 `android_ripple` no iPhone: o que acontece?

PARTE 2

FLATLIST – SÓ RENDERIZA O QUE SE VÊ

VOCÊ JÁ SABE MONTAR UMA LISTA

```
// ❌ Funciona com 5 itens. Trava com 5 000.  
<ScrollView>  
  {habitots.map((h) => <CardHabito key={h.id} habito={h} />)}  
</ScrollView>
```

ISSO NÃO ESTÁ ERRADO POR PRINCÍPIO.

A `ScrollView` cumpre o contrato dela: **montar todos os filhos de uma vez.**

O erro é usá-la para conteúdo que **cresce sem limite.**

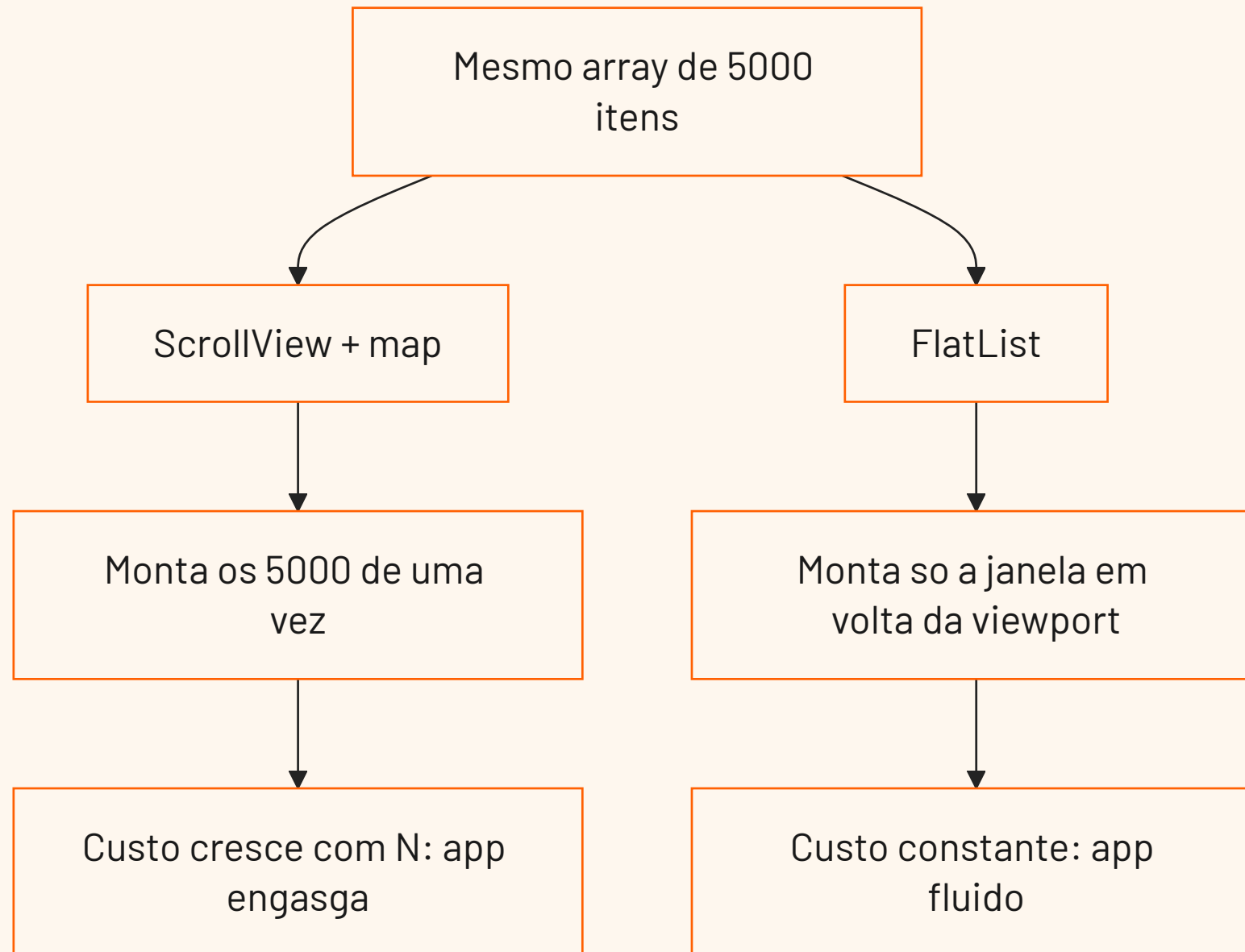
A JANELA DO TREM

A PAISAGEM TEM 300 KM. A JANELA TEM UM METRO.

O trem não carrega a paisagem inteira — mostra o pedaço que passa, mais um pouco antes e um pouco depois.

Acelerou demais? Você vê **borrão**.

O QUE MUDA





O VOCABULÁRIO OFICIAL

Termo	O que é
Viewport	a área visível na tela
Janela (window)	a área em que os itens ficam montados — bem maior
Área em branco	o vazio quando você rola mais rápido do que ela renderiza
VirtualizedList	o componente por baixo do FlatList e do SectionList

COM ESSES QUATRO TERMOS VOCÊ **DESCREVE** O BUG, EM VEZ DE DIZER "ESTÁ LENTO".



O MÍNIMO FUNCIONAL

```
// Declarada FORA do componente: criada uma vez, não a cada render.  
function renderHabito({ item }: { item: Habito }) {  
  return (  
    <View style={styles.item}>  
      <Text style={styles.titulo}>{item.titulo}</Text>  
      <Text style={styles.legenda}>{item.categoria} · {item.streakDias} dias</Text>  
    </View>  
  );  
}  
  
<FlatList style={styles.lista} data={HABITOS} renderItem={renderHabito} />
```

O CORPO DO `renderItem` É O MESMO QUE IA DENTRO DO `map` .



"SEMPRE DEFINA `keyExtractor`."

NÃO. A DOCUMENTAÇÃO DIZ O CONTRÁRIO.

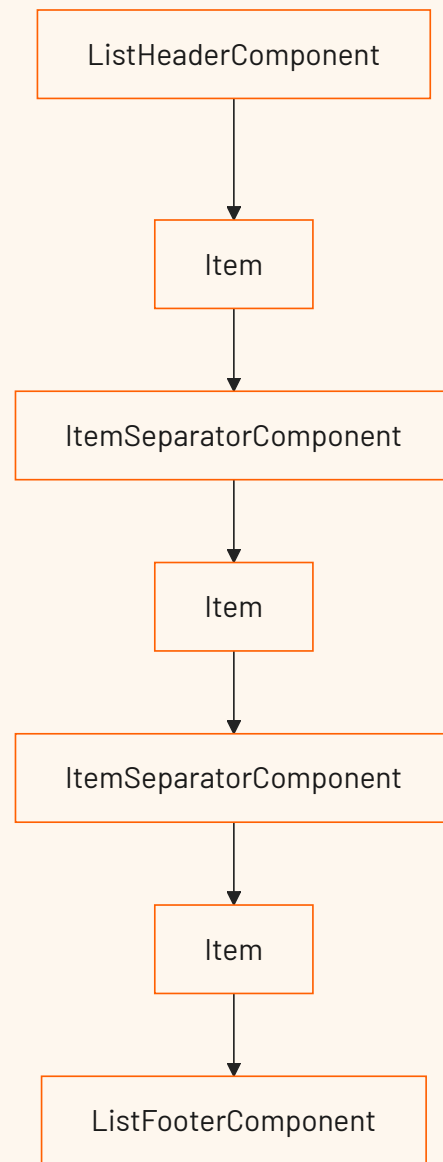
O extractor padrão checa `item.key`, depois `item.id`, e só então o índice.

ENTÃO QUANDO ESCREVER `keyExtractor` ?

```
<FlatList
  data={habitots}
  renderItem={renderHabito}
  keyExtractor={(item) => item.codigo} // não é `key` nem `id`
/>
```

O que é **de fato** errado: cair no **índice** numa lista que reordena, filtra ou remove.

AS MOLDURAS DA LISTA



ListEmptyComponent:
ocupa o lugar dos itens
quando data esta vazio

AS MOLDURAS — PARA QUE SERVE CADA UMA

Prop	Para quê
ListHeaderComponent	título, busca, filtros — rola junto
ListFooterComponent	total, aviso de fim, "carregando mais"
ListEmptyComponent	estado vazio
ItemSeparatorComponent	a linha entre itens
contentContainerStyle	padding e gap do conteúdo que rola



LISTA VAZIA É ESTADO DE PRODUTO, NÃO CASO DE BORDA.

PRIMEIRO ACESSO · FILTRO SEM RESULTADO · BUSCA SEM MATCH

Uma tela em branco parece **bug** para quem está usando.

DOIS ERROS NAS MOLDURAS

```
// ✗ sobra margem depois do último item
item: { paddingVertical: 12, marginBottom: 8 }

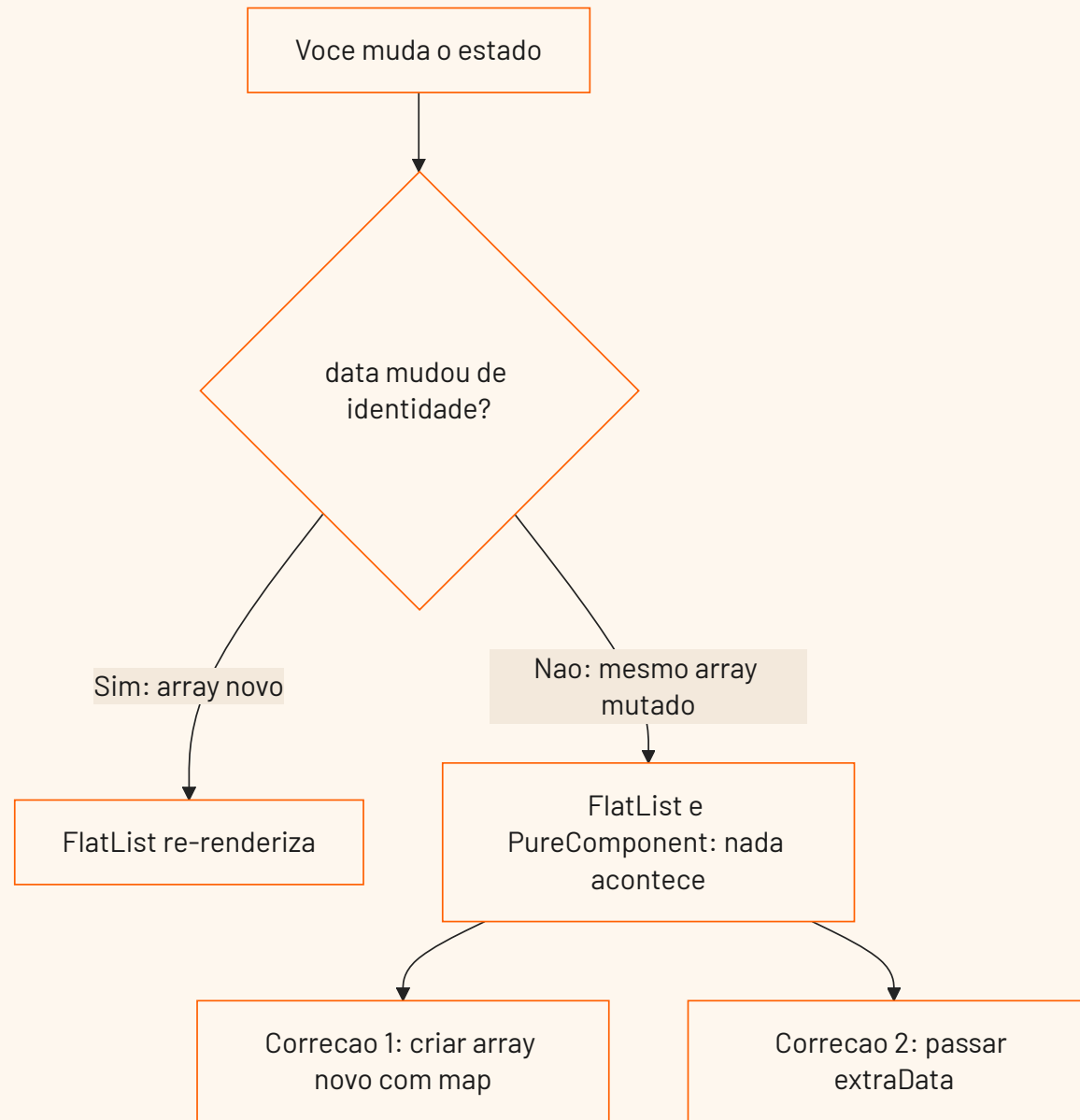
// ✓ o separador não existe nas pontas
ItemSeparatorComponent={Separador}

// ✓ ou: gap no contêiner (é uma View flex – Aula 3)
contentContainerStyle={{ gap: 8, padding: 16 }}
```

```
// ✗ tipo de componente novo a cada render → remonta
<FlatList ListHeaderComponent={() => <Cabecalho />} />

// ✓ passe o elemento, ou uma referência estável
<FlatList ListHeaderComponent={<Cabecalho />} />
```

A LISTA QUE NÃO ATUALIZA



MUTAR × CRIAR

```
// ❌ mudou: `habitos` continua sendo o MESMO array. A lista não muda.  
const alvo = hábitos.find((h) => h.id === id);  
if (alvo) alvo.status = 'concluído';  
setHábitos(habitos);  
  
// ✅ array novo, objeto novo: a identidade muda, a lista re-renderiza  
setHábitos((atuais) =>  
  atuais.map((h) => (h.id === id ? { ...h, status: 'concluído' } : h)),  
);
```

E QUANDO O DADO QUE MUDA NÃO ESTÁ EM `data` ? AÍ SIM: `extraData` .

A PERGUNTA QUE DECIDE:

ISSO É **DADO DO DOMÍNIO** OU **ESTADO DE TELA**?

Dado do domínio → imutabilidade. · Estado de tela → `extraData`.

COLUNAS, CARROSSEL E RECARREGAR

```
<FlatList numColumns={2} columnWrapperStyle={{ gap: 12 }} ... />
```

- `numColumns` – só com `horizontal={false}`, e os itens precisam ter **a mesma altura**
- `horizontal` – carrossel
- `refreshing` + `onRefresh` – puxar para atualizar, sem componente extra
- `onEndReached` + `onEndReachedThreshold` – rolagem infinita

HOJE, COM ARRAY LOCAL. BUSCAR DE UMA API É ASSUNTO DAS AULAS 10 A 12.

 ERRO 1: FlatList DENTRO DE ScrollView

VirtualizedLists should never be nested inside plain ScrollViews with the same orientation because it can break windowing and other functionality – use another VirtualizedList-backed container instead.

Por que acontece: você quer que "a tela inteira role".

Por que é grave: a `ScrollView` dá altura infinita à lista → a janela deixa de fazer sentido → **tudo** é montado.

VOCÊ REINTRODUZIU EXATAMENTE O PROBLEMA QUE VEIO RESOLVER.

ERRO 2: renderItem ANÔNIMO NO JSX

```
// ❌ função nova a cada render
<FlatList renderItem={({ item }) => <ItemHabitto habitto={item} />} />

// ✅ declarada fora do componente: criada uma vez só
function renderHabitto({ item }: { item: Habitto }) { ... }
<FlatList renderItem={renderHabitto} />
```

 **Nota honesta:** existe uma ferramenta do React feita exatamente para isso. Ela **não é assunto de hoje** — chega na **Aula 6, sobre Hooks**.

ERRO 3: LISTA SEM ALTURA

```
// ❌ nada aparece
<View>
  <FlatList data={habitots} renderItem={renderHabito} />
</View>

// ✅
<View style={{ flex: 1 }}>
  <FlatList data={habitots} renderItem={renderHabito} />
</View>
```

MESMA REGRA DA AULA 3: **FLEXBOX DISTRIBUI ESPAÇO QUE EXISTE.**

OS BOTÕES DE AJUSTE

Prop	Default	O que faz
<code>initialNumToRender</code>	<code>10</code>	itens no primeiro lote
<code>windowSize</code>	<code>21</code>	tamanho da janela, em múltiplos da viewport
<code>maxToRenderPerBatch</code>	<code>10</code>	itens por lote a cada rolagem
<code>updateCellsBatchingPeriod</code>	<code>50 ms</code>	intervalo entre lotes
<code>removeClippedSubviews</code>	<code>true</code> no Android	desanexa views fora da viewport
<code>getItemLayout</code>	—	dispensa a medição, se a altura é fixa

A REGRA: **NÃO MEXA ANTES DE MEDIR.** TODO AJUSTE É UMA TROCA.

 removeClippedSubviews NÃO É OTIMIZAÇÃO GRÁTIS

A própria documentação avisa:

"this implementation can have bugs, such as missing content (mainly observed on iOS)... this does not save significant memory because the views are not deallocated, only detached."

CONTEÚDO SUMINDO NO IOS. E A ECONOMIA DE MEMÓRIA QUE NÃO EXISTE.

PARTE 3

SECTIONLIST – QUANDO O DADO VEM AGRUPADO

O CRITÉRIO, EM UMA PERGUNTA

OS MEUS DADOS TÊM GRUPOS COM TÍTULO?

Sim → `SectionList` · **Não** → `FlatList`

Hábitos por período · contatos por letra · tarefas por status · mensagens por data

O que **não** justifica trocar: querer um título no topo. Isso é `ListHeaderComponent` .

O FORMATO DE sections

```
const SECOES = [  
  { title: 'Manhã', data: [habito1, habito2] },  
  { title: 'Noite', data: [habito3] },  
];
```

● "A seção precisa ter `title` ." **Não precisa.**

A documentação define **data** como o campo obrigatório. `title` é um campo **seu**.



CABEÇALHOS, RODAPÉS E OS DOIS SEPARADORES

Componente	Onde aparece
renderSectionHeader	no topo de cada seção
renderSectionFooter	no fim de cada seção
ItemSeparatorComponent	entre itens, nunca nas pontas
SectionSeparatorComponent	no topo e no fim de cada seção

Item... CUIDA DO ESPAÇO DENTRO DO GRUPO. **Section...** , EM VOLTA DELE.



O BUG "FUNCIONA NO MEU CELULAR"

STICKYSECTIONHEADERSENABLED

IOS: DEFAULT `true` · ANDROID: DEFAULT `false`

A CORREÇÃO: DECLARAR

```
<SectionList
  sections={SECOES}
  renderItem={renderHabito}
  stickySectionHeadersEnabled
/>
```

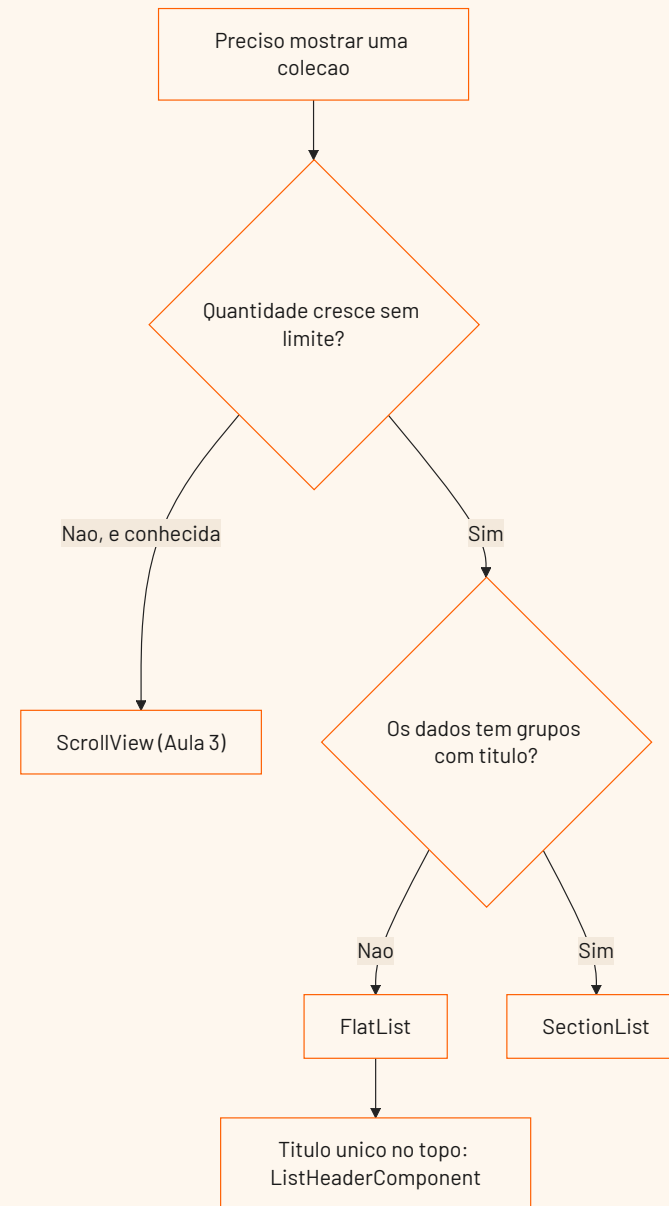
DEFAULT DE PLATAFORMA É **DECISÃO DE PRODUTO DISFARÇADA DE OMISSÃO.**

DE DADOS PLANOS PARA SEÇÕES

```
function agruparPorStatus(habitos: Habito[]): Secao[] {  
  const rotulos: Record<StatusHabito, string> = {  
    pendente: 'Pendentes', concluido: 'Concluídos', pulado: 'Pulados',  
  };  
  const ordem: StatusHabito[] = ['pendente', 'concluido', 'pulado'];  
  
  return ordem  
    .map((s) => ({ title: rotulos[s], data: hábitos.filter((h) => h.status === s) })))  
    .filter((secao) => secao.data.length > 0); // não mostrar grupo vazio  
}
```

JAVASCRIPT COMUM. O SECTIONLIST NÃO AGRUPA NADA.

A ÁRVORE DE DECISÃO



FlatList × SectionList

	FlatList	SectionList
Dados	<code>data</code> (array)	<code>sections</code> (array de <code>{ data }</code>)
Cabeçalho por grupo	não tem	<code>renderSectionHeader</code>
<code>renderItem</code> recebe	<code>{ item, index, separators }</code>	<code>{ item, index, section, separators }</code>
Sticky header	—	<code>stickySectionHeadersEnabled</code>
Por baixo	<code>VirtualizedList</code>	<code>VirtualizedList</code>

`keyExtractor` , `SEPARADORES`, `VAZIO`, `extraData` , `onRefresh` : **IGUAIS NOS DOIS.**



E AS BIBLIOTECAS DE LISTA? (CONTEXTO)

Existem bibliotecas de terceiros focadas em desempenho de lista.

A DISCIPLINA ENSINA O QUE VEM NO FRAMEWORK.

Dois motivos: é o que você encontra em qualquer código base, e virtualização, janela, `keyExtractor` e imutabilidade são os **mesmos conceitos** em todas elas.





CHECKPOINT 2

60 SEGUNDOS EM DUPLAS.



CHECKPOINT 2 – AS PERGUNTAS

- 1 12 campos num formulário: `ScrollView` ou `FlatList` ? Justifique pelo **contrato**.
- 2 Mudei o estado e a lista não atualizou. **Duas** causas e as correções.
- 3 `keyExtractor` devolvendo índice: em que situação isso quebra?
- 4 O cabeçalho gruda no iPhone da sua dupla e não no seu Android. Quem está errado?

PARTE 4

HANDS-ON – 38 MINUTOS

O ROTEIRO DO HANDS-ON

#	O quê	Tempo
1 e 2	Associação + caça ao erro — sem computador	8 min
3	<code>BotaoAcao</code> com <code>Pressable</code> , <code>pressed</code> , <code>hitSlop</code> , ripple	8 min
4	Previsão do ciclo do toque — prever antes de rodar	5 min
5	<code>FlatList</code> completa: chave, separador, vazio, cabeçalho	10 min
6	<code>SectionList</code> de dados planos + sticky nas duas plataformas	7 min
7	(sobra) a lista que não atualiza	casa

DOMÍNIO HÁBITO → `exercises.md` • DOMÍNIO PET → `practice.md`



ATIVIDADE APLICADA — A TELA DE LISTA DE VERDADE

SectionList agrupando por status · itens tocáveis com **Pressable** · ação secundária no **toque longo**

OS **QUATRO** ESTADOS DA LISTA, TODOS IMPLEMENTADOS:

- 1 **com dados** — o caso normal
- 2 **vazio** — com texto que diga o que fazer
- 3 **atualizando** — `refreshing` + `onRefresh`
- 4 **filtrado sem resultado** — e o texto **não** pode ser o mesmo do estado 2

MÍNIMO DE 20 ITENS NO MOCK. LISTA DE 3 NÃO PROVA NADA.

VOCÊ ENTROU COM UMA TELA BONITA QUE SÓ SABIA MOSTRAR TRÊS ITENS.

SAI COM UM TOQUE DE VERDADE E UMA LISTA CUJO CUSTO NÃO DEPENDE DO TAMANHO DOS DADOS.

Aula 5: sensores do dispositivo — e o dado da lista deixa de ser mock.

ATALHOS DESTA APRESENTAÇÃO

Tecla	Ação
→ / espaço · ←	avançar · voltar
S	modo apresentador — notas, cronômetro e próximo slide
F	tela cheia
0 ou Esc	visão geral de todos os slides
B ou .	tela preta
?	lista completa de atalhos